Chapter 10. Testing: Evaluation, Monitoring, and Continuous Improvement

In Chapter 9, you learned how to deploy your AI application into production and utilize LangGraph Platform to host and debug your app.

Although your app can respond to user inputs and execute complex tasks, its underlying LLM is nondeterministic and prone to hallucination. As discussed in previous chapters, LLMs can generate inaccurate and outdated outputs due to a variety of reasons including the prompt, format of user's input, and retrieved context. In addition, harmful or misleading LLM outputs can significantly damage a company's brand and customer loyalty.

To combat this tendency toward hallucination, you need to build an efficient system to test, evaluate, monitor, and continuously improve your LLM applications' performance. This robust testing process will enable you to quickly debug and fix AI-related issues before and after your app is in production.

In this chapter, you'll learn how to build an iterative testing system across the key stages of the LLM app development life-cycle and maintain high performance of your application.

Testing Techniques Across the LLM App Development Cycle

Before we construct the testing system, let's briefly review how testing can be applied across the three key stages of LLM app development:

Design

In this stage, LLM tests are applied directly to your application. These tests can be assertions executed at runtime that feed failures back to the LLM for self-correction. The purpose of testing at this stage is error handling within your app before it affects users.

Preproduction

In this stage, tests are run right before deployment into production. The purpose of testing at this stage is to catch and fix any regressions before the app is released to real users.

Production

In this stage, tests are run while your application is in production to help monitor and catch errors affecting real users. The purpose is to identify issues and feed them back into the design or preproduction phases.

The combination of testing across these stages creates a continuous improvement cycle where these steps are repeated: design, test, deploy, monitor, fix, and redesign. See Figure 10-1.

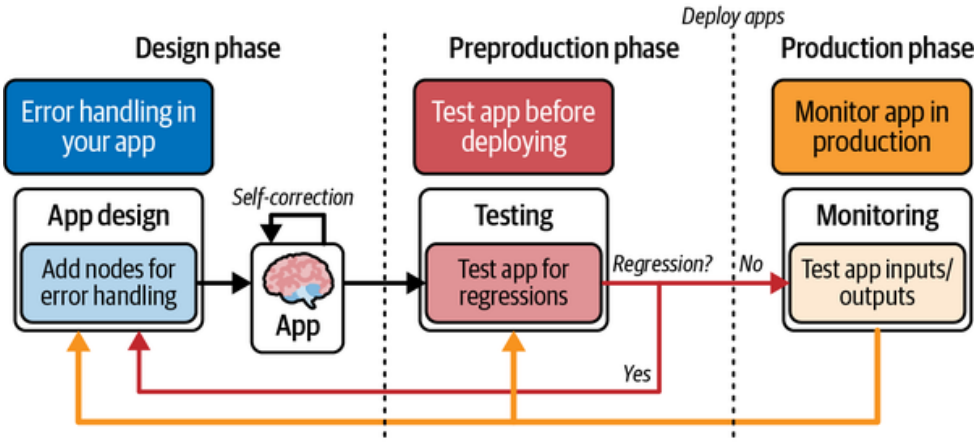

Figure 10-1. The three key stages of the LLM app development cycle

In essence, this cycle helps you to identify and fix production issues in an efficient and quick manner.

Let's dive deeper into testing techniques across each of these stages.

The Design Stage: Self-Corrective RAG

As discussed previously, your application can incorporate error handling at runtime that feeds errors to the LLM for self-correction. Let's explore a RAG use case using LangGraph as the framework to orchestrate error handling.

Basic RAG-driven AI applications are prone to hallucination due to inaccurate or incomplete retrieval of relevant context to generate outputs. But you can utilize an LLM to grade retrieval relevance and fix hallucination issues.

LangGraph enables you to effectively implement the control flow of this process, as shown in Figure 10-2.

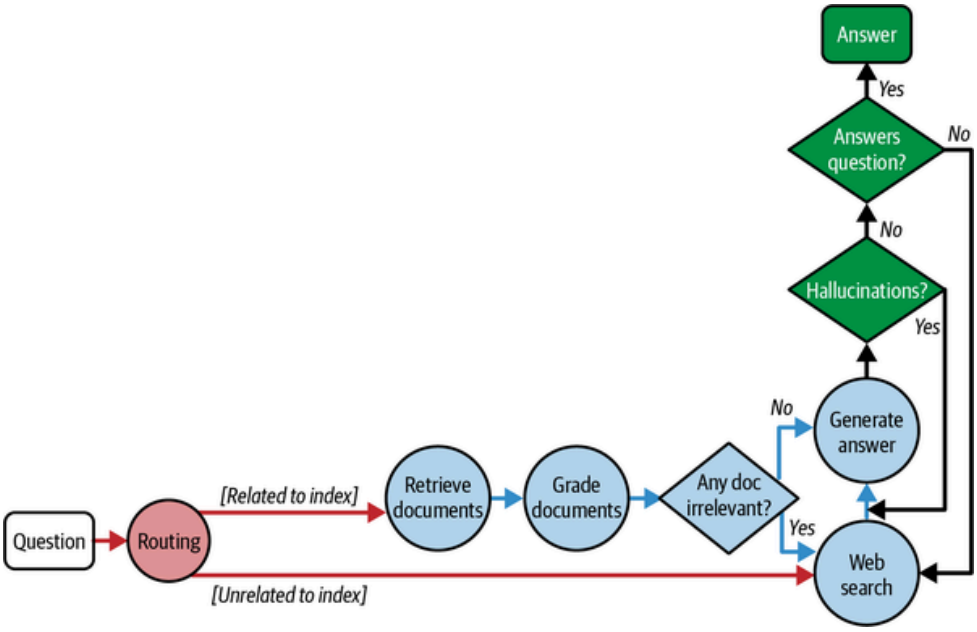

Figure 10-2. Self-corrective RAG control flow

The control flow steps are as follows:

1. In the routing step, each question is routed to the relevant retrieval method, that is, vector store and web search.
2. If, for example, the question is routed to a vector store for retrieval, the LLM in the control flow will retrieve and grade the documents for relevancy.
3. If the document is relevant, the LLM proceeds to generate an answer.
4. The LLM will check the answer for hallucinations and only proceed to display the answer to the user if the output is accurate and relevant.
5. As a fallback, if the retrieved document is irrelevant or the generated answer doesn't answer the user's question, the flow utilizes web search to retrieve relevant information as context.

This process enables your app to iteratively generate answers, self-correct errors and hallucinations, and improve the quality of outputs.

Let's run through an example code implementation of this control flow. First, download the required packages and initialize relevant API keys. For these examples, you'll need to set your OpenAI and LangSmith API keys as environment variables.

First, we'll create an index of three blog posts:

Python

```
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain_community.document_loaders import WebBaseLoader
from langchain_community.vectorstores import InMemoryVectorStore
from langchain_openai import OpenAIEmbeddings
from langchain_core.prompts import ChatPromptTemplate
from pydantic import BaseModel, Field
```

```python
from langchain_openai import ChatOpenAI


# --- Create an index of documents ---

urls = [
    "https://blog.langchain.dev/top-5-langgraph-agents-in-production-2024/
    "https://blog.langchain.dev/langchain-state-of-ai-2024/",
    "https://blog.langchain.dev/introducing-ambient-agents/",
]

docs = [WebBaseLoader(url).load() for url in urls]
docs_list = [item for sublist in docs for item in sublist]

text_splitter = RecursiveCharacterTextSplitter.from_tiktoken_encoder(
    chunk_size=250, chunk_overlap=0
)
doc_splits = text_splitter.split_documents(docs_list)

# Add to vectorDB
vectorstore = InMemoryVectorStore.from_documents(
    documents=doc_splits,
    embedding=OpenAIEmbeddings(),
)
retriever = vectorstore.as_retriever()

# Retrieve the relevant documents
results = retriever.invoke(
    "What are 2 LangGraph agents used in production in 2024?")

print("Results: \n", results)
```

JavaScript

```javascript
import { RecursiveCharacterTextSplitter } from '@langchain/textsplitters';
import {
  CheerioWebBaseLoader
} from "@langchain/community/document_loaders/web/cheerio";
import {
  InMemoryVectorStore
} from '@langchain/community/vectorstores/in_memory';
import { OpenAIEmbeddings } from '@langchain/openai';
import { ChatPromptTemplate } from '@langchain/core/prompts';
import { z } from 'zod';
import { ChatOpenAI } from '@langchain/openai';

const urls = [
  'https://blog.langchain.dev/top-5-langgraph-agents-in-production-2024/',
  'https://blog.langchain.dev/langchain-state-of-ai-2024/',
  'https://blog.langchain.dev/introducing-ambient-agents/',
];
```

```
// Load documents from URLs
const loadDocs = async (urls) => {
  const docs = [];
  for (const url of urls) {
    const loader = new CheerioWebBaseLoader(url);
    const loadedDocs = await loader.load();
    docs.push(...loadedDocs);
  }
  return docs;
};

const docsList = await loadDocs(urls);

// Initialize the text splitter
const textSplitter = new RecursiveCharacterTextSplitter({
  chunkSize: 250,
  chunkOverlap: 0,
});

// Split the documents into smaller chunks
const docSplits = textSplitter.splitDocuments(docsList);

// Add to vector database
const vectorstore = await InMemoryVectorStore.fromDocuments(
  docSplits,
  new OpenAIEmbeddings()
);

// The `retriever` object can now be used for querying
const retriever = vectorstore.asRetriever();

const question = 'What are 2 LangGraph agents used in production in 2024?'

const docs = retriever.invoke(question);

console.log('Retrieved documents: \n', docs[0].page_content);
```

As discussed previously, the LLM will grade the relevancy of the retrieved documents from the index. We can construct this instruction in a system prompt:

Python

```
### Retrieval Grader

from langchain_core.prompts import ChatPromptTemplate
from langchain_core.pydantic_v1 import BaseModel, Field
from langchain_openai import ChatOpenAI


# Data model
```

```python
class GradeDocuments(BaseModel):
    """Binary score for relevance check on retrieved documents."""

    binary_score: str = Field(
        description="Documents are relevant to the question, 'yes' or 'no'"
    )


# LLM with function call
llm = ChatOpenAI(model="gpt-3.5-turbo", temperature=0)
structured_llm_grader = llm.with_structured_output(GradeDocuments)

# Prompt
system = """You are a grader assessing relevance of a retrieved document to a
    user question.
    If the document contains keyword(s) or semantic meaning related to the
    question, grade it as relevant.
    Give a binary score 'yes' or 'no' to indicate whether the document is
    relevant to the question."""
grade_prompt = ChatPromptTemplate.from_messages(
    [
        ("system", system),
        ("human", """Retrieved document: \n\n {document} \n\n User question:
            {question}"""),
    ]
)

retrieval_grader = grade_prompt | structured_llm_grader
question = "agent memory"
docs = retriever.get_relevant_documents(question)
doc_txt = docs[0].page_content # as an example
retrieval_grader.invoke({"question": question, "document": doc_txt})
```

JavaScript

```javascript
import { ChatPromptTemplate } from "@langchain/core/prompts";
import { z } from "zod";
import { ChatOpenAI } from "@langchain/openai";

// Define the schema using Zod
const GradeDocumentsSchema = z.object({
  binary_score: z.string().describe(`Documents are relevant to the question,
      'yes' or 'no'`),
});

// Initialize LLM with structured output using Zod schema
const llm = new ChatOpenAI({ model: "gpt-3.5-turbo", temperature: 0 });
const structuredLLMGrader = llm.withStructuredOutput(GradeDocumentsSchema);

// System and prompt template
const systemMessage = `You are a grader assessing relevance of a retrieved
  document to a user question.
```

```
  If the document contains keyword(s) or semantic meaning related to the
    question, grade it as relevant.
  Give a binary score 'yes' or 'no' to indicate whether the document is relev
    to the question.`;

  const gradePrompt = ChatPromptTemplate.fromMessages([
    { role: "system", content: systemMessage },
    {
      role: "human",
      content: "Retrieved document: \n\n {document} \n\n
        User question: {question}",
    },
  ]);

  // Combine prompt with the structured output
  const retrievalGrader = gradePrompt.pipe(structuredLLMGrader);

  const question = "agent memory";
  const docs = await retriever.getRelevantDocuments(question);

  await retrievalGrader.invoke({
    question,
    document: docs[1].pageContent,
  });
```

The output:

```
binary_score='yes'
```

Notice the use of Pydantic/Zod to help model the binary decision output in a format that can be used to programmatically decide which node in the control flow to move toward.

In LangSmith, you can see a trace of the logic flow across the nodes discussed previously (see Figure 10-3).

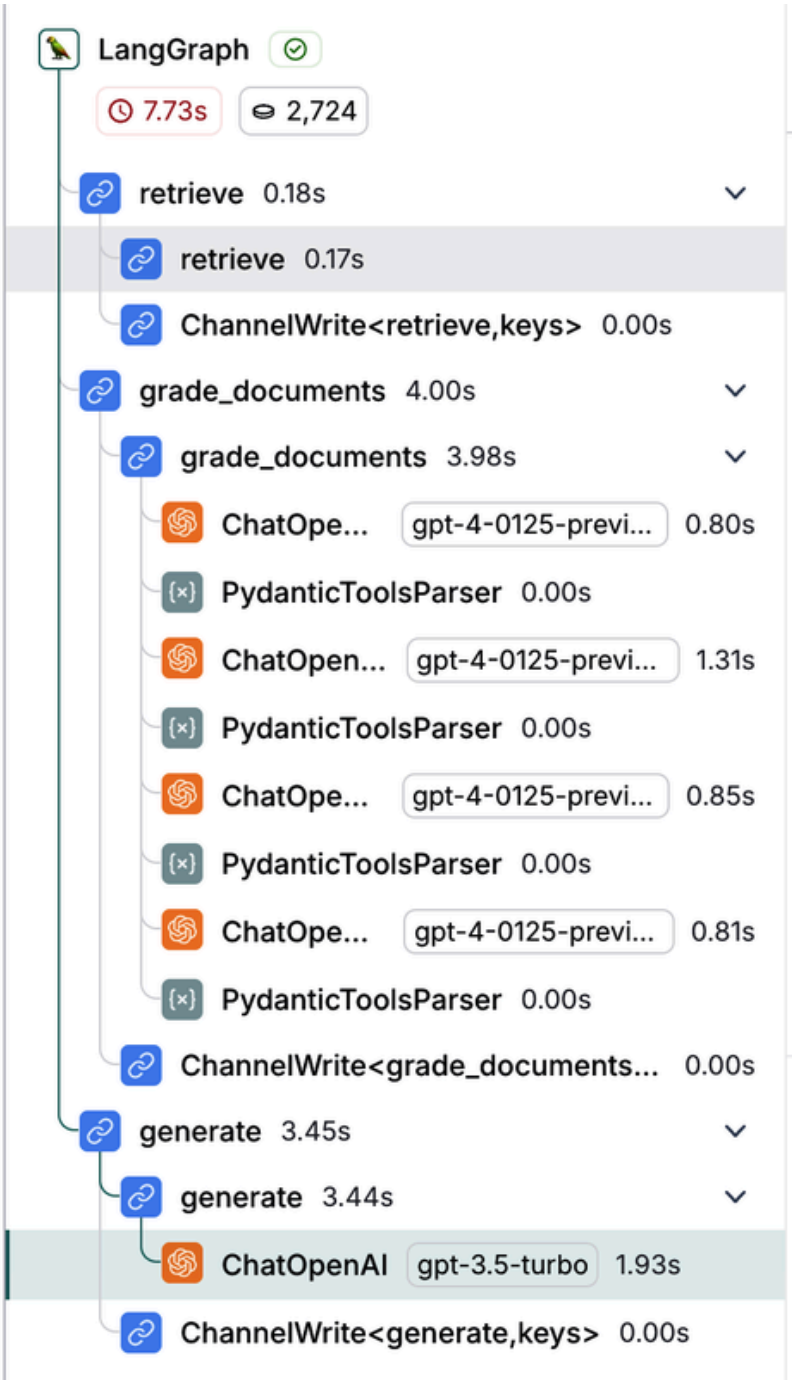

Figure 10-3. LangSmith trace results

Let's test to see what happens when the input question cannot be answered by the retrieved documents in the index.

First, utilize LangGraph to make it easier to construct, execute, and debug the full control flow. See the full graph definition in the book's GitHub repository. Notice that we've added a `transform_query` node to help rewrite the input query in a format that web search can use to retrieve higher-quality results.

As a final step, we set up our web search tool and execute the graph using the out-of-context question. The LangSmith trace shows that the web search tool was used as a fallback to retrieve relevant information prior to the final LLM generated answer (see Figure 10-4).

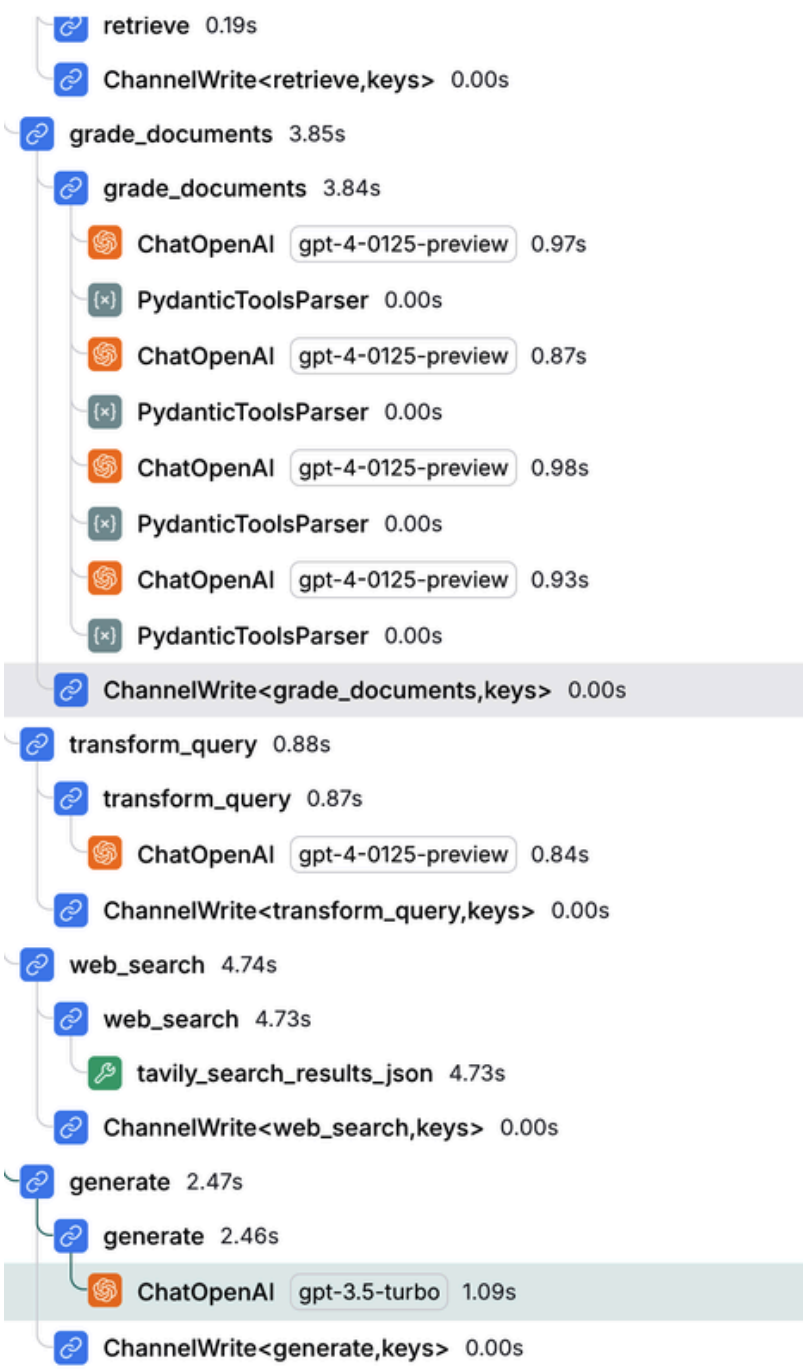

Figure 10-4. LangSmith trace of self-corrective RAG utilizing web search as a fallback

Let's move on to the next stage in LLM app testing: preproduction.

The Preproduction Stage

The purpose of the preproduction stage of testing is to measure and evaluate the performance of your application prior to production. This will enable you to efficiently assess the accuracy, latency, and cost of utilizing the LLM.

Creating Datasets

Prior to testing, you need to define a set of scenarios you'd like to test and evaluate. A *dataset* is a collection of examples that provide inputs and expected outputs used to evaluate your LLM app.

These are three common methods to build datasets for valuation:

Manually curated examples

These are handwritten examples based on expected user inputs and ideal generated outputs. A small dataset consists of between 10 and 50 quality examples. Over time, more examples can be added to the dataset based on edge cases that emerge in production.

Application logs

Once the application is in production, you can store real-time user inputs and later add them to the dataset. This will help ensure the dataset is realistic and covers the most common user questions.

Synthetic data

These are artificially generated examples that simulate various scenarios and edge cases. This enables you to generate new inputs by sampling existing inputs, which is useful when you don't have enough real data to test on.

In LangSmith, you can create a new dataset by selecting Datasets and Testing in the sidebar and clicking the "+ New Dataset" button on the top right of the app, as shown in Figure 10-5.

In the opened window, enter the relevant dataset details, including a name, description, and dataset type. If you'd like to use your own dataset, click the "Upload a CSV dataset" button.

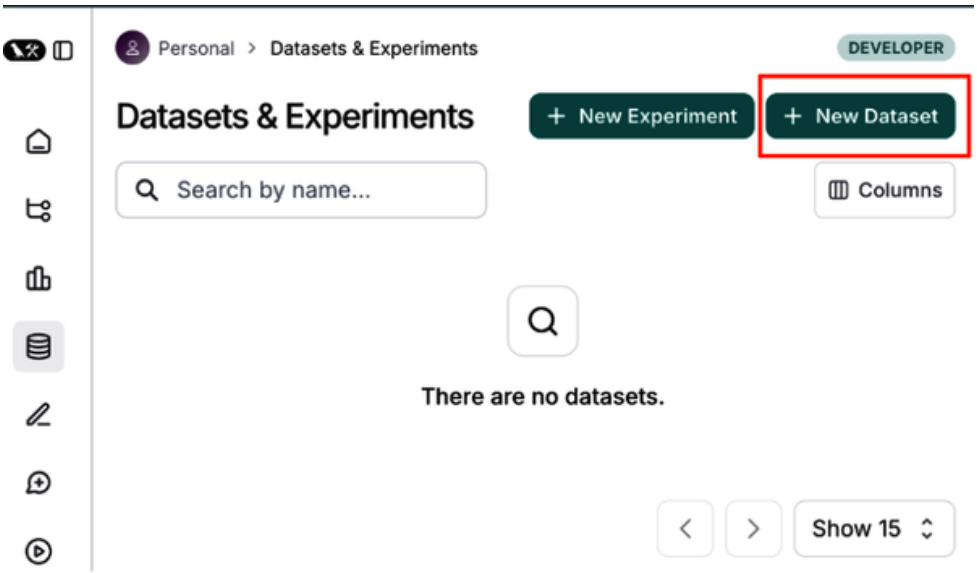

Figure 10-5. Creating a new dataset in the LangSmith UI

LangSmith offers three different dataset types:

`kv` *(key-value) dataset*

- *Inputs* and *outputs* are represented as arbitrary key-value pairs.
- The `kv` dataset is the most versatile, and it is the default type. The `kv` dataset is suitable for a wide range of evaluation scenarios.
- This dataset type is ideal for evaluating chains and agents that require multiple inputs or generate multiple outputs.

`llm` (large language model) dataset

- The `llm` dataset is designed for evaluating completion style language models.
- The inputs dictionary contains a single input key mapped to the prompt string.
- The outputs dictionary contains a single output key mapped to the corresponding response string.
- This dataset type simplifies evaluation for LLMs by providing a standardized format for inputs and outputs.

`chat` dataset

- The `chat` dataset is designed for evaluating LLM structured chat messages as inputs and outputs.
- The *inputs* dictionary contains a single *input* key mapped to a list of serialized chat messages.
- The *outputs* dictionary contains a single *output* key mapped to a list of serialized chat messages.
- This dataset type is useful for evaluating conversational AI systems or chatbots.

The most flexible option is the key-value data type (see Figure 10-6).

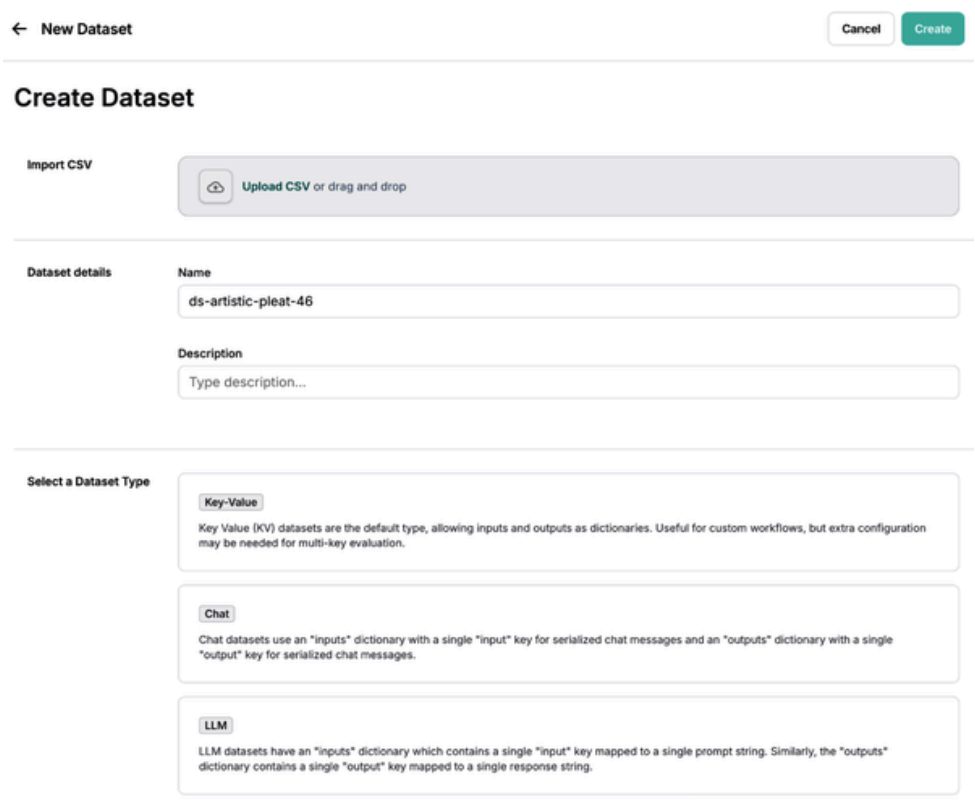

Figure 10-6. Selecting a dataset type in the LangSmith UI

Next, add examples to the dataset by clicking Add Example. Provide the input and output examples as JSON objects, as shown in Figure 10-7.

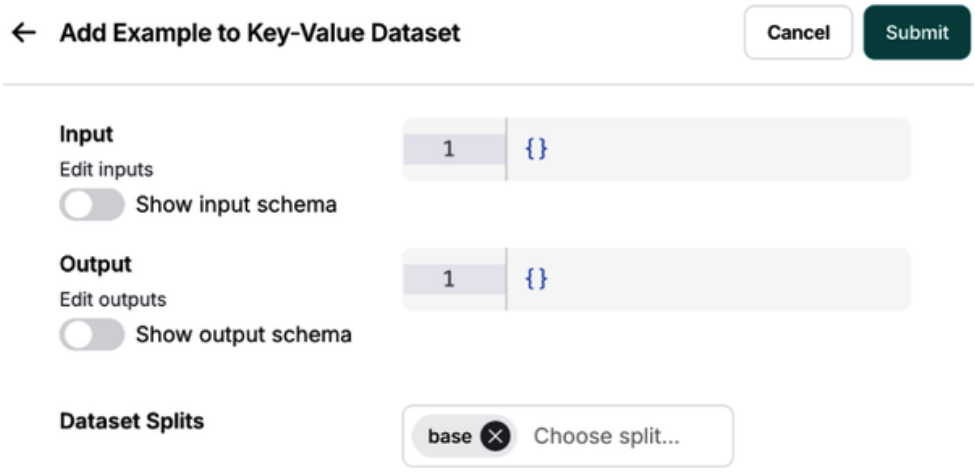

Figure 10-7. Add key-value dataset examples in the LangSmith UI

You can also define a schema for your dataset in the “Dataset schema” section, as shown in Figure 10-8.

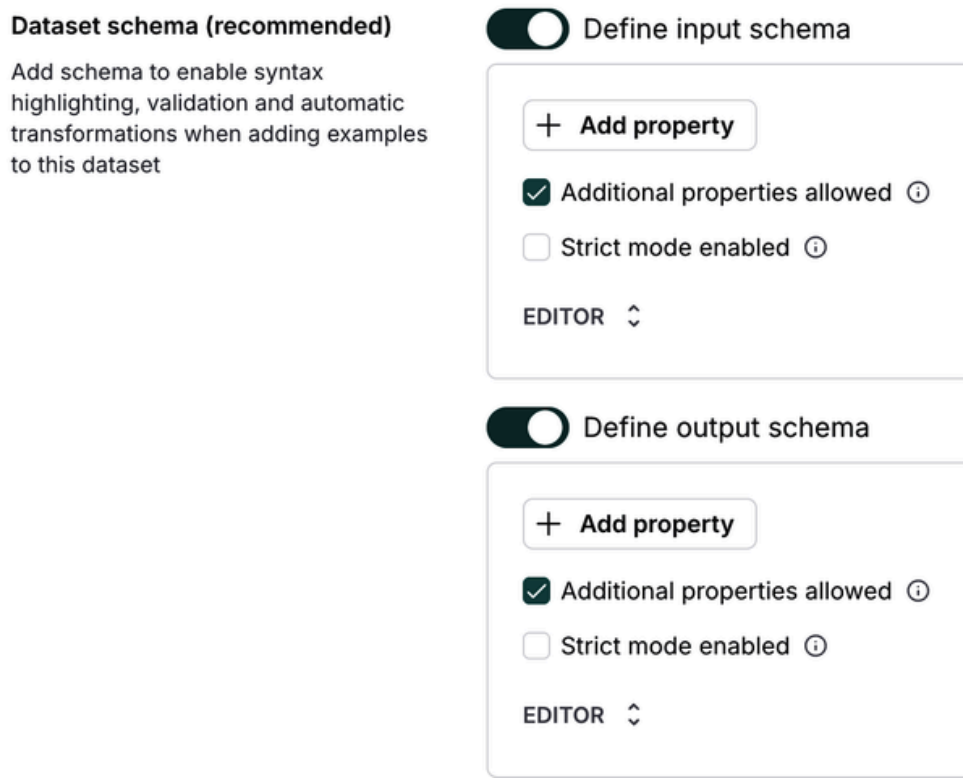

Figure 10-8. Adding a dataset schema in the LangSmith UI

Defining Your Evaluation Criteria

After creating your dataset, you need to define evaluation metrics to assess your application's outputs before deploying into production. This batch evaluation on a predetermined test suite is often referred to as *offline evaluation*.

For offline evaluation, you can optionally label expected outputs (that is, ground truth references) for the data points you are testing on. This enables you to compare your application's response with the ground truth references, as shown in Figure 10-9.

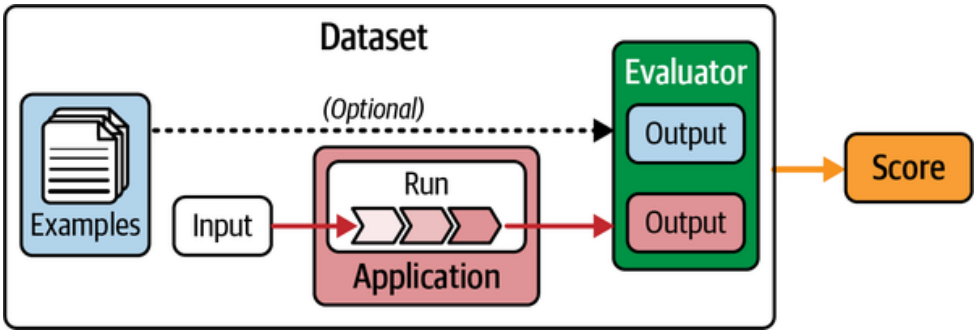

Figure 10-9. AI evaluation diagram

There are three main evaluators to score your LLM app performance:

Human evaluators

If you can't express your testing requirements as code, you can use human feedback to express qualitative characteristics and label

app responses with scores. LangSmith speeds up the process of collecting and incorporating human feedback with annotation queues.

Heuristic evaluators

These are hardcoded functions and assertions that perform computations to determine a score. You can use reference-free heuristics (for example, checking whether output is valid JSON) or reference-based heuristics such as accuracy. Reference-based evaluation compares an output to a predefined ground truth, whereas reference-free evaluation assesses qualitative characteristics without a ground truth. Custom heuristic evaluators are useful for code-generation tasks such as schema checking and unit testing with hard-coded evaluation logic.

LLM-as-a-judge evaluators

This evaluator integrates human grading rules into an LLM prompt to evaluate whether the output is correct relative to the reference answer supplied from the dataset output. As you iterate in preproduction, you'll need to audit the scores and tune the LLM-as-a-judge to produce reliable scores.

To get started with evaluation, start simple with heuristic evaluators. Then implement human evaluators before moving on to LLM-as-a-judge to automate your human review. This enables you to add depth and scale once your criteria are well-defined.

TIP

When using LLM-as-a-judge evaluators, use straightforward prompts that can easily be replicated and understood by a human. For example, avoid asking an LLM to produce scores on a range of 0 to 10 with vague distinctions between scores.

Figure 10-10 illustrates LLM-as-a-judge evaluator in the context of a RAG use case. Note that the reference answer is the ground truth.

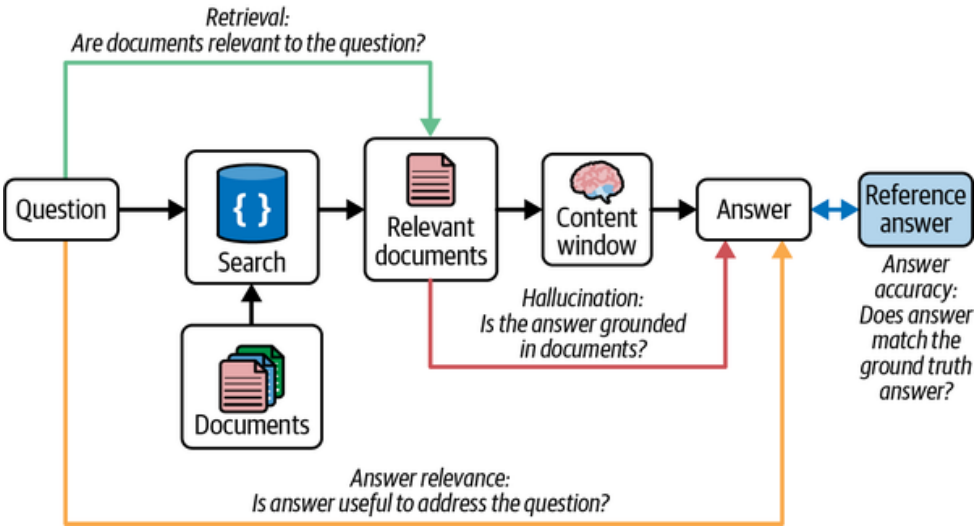

Figure 10-10. LLM-as-a-judge evaluator used in a RAG use case

Improving LLM-as-a-judge evaluators performance

Using an LLM-as-a-judge is an effective method to grade natural language outputs from LLM applications. This involves passing the generated output to a separate LLM for judgment and evaluation. But how can you trust the results of LLM-as-a-judge evaluation?

Often, rounds of prompt engineering are required to improve accuracy, which is cumbersome and time-consuming. Fortunately, LangSmith provides a *few-shot* prompt solution whereby human corrections to LLM-as-a-judge outputs are stored as few-shot examples, which are then fed back into the prompt in future iterations.

By utilizing few-shot learning, the LLM can improve accuracy and align outputs with human preferences by providing examples of correct behavior. This is especially useful when it's difficult to construct instructions on how the LLM should behave or be formatted.

The few-shot evaluator follows these steps:

1. The LLM evaluator provides feedback on generated outputs, assessing factors such as correctness, relevance, or other criteria.
2. It adds human corrections to modify or correct the LLM evaluator's feedback in LangSmith. This is where human preferences and judgment are captured.
3. These corrections are stored as few-shot examples in LangSmith, with an option to leave explanations for corrections.
4. The few-shot examples are incorporated into future prompts as subsequent evaluation runs.

Over time, the few-shot evaluator will become increasingly aligned with human preferences. This self-improving mechanism reduces the need for time-consuming prompt engineering, while improving the accuracy and relevance of LLM-as-a-judge evaluations.

Here's how to easily set up the LLM-as-a-judge evaluator in LangSmith for offline evaluation. First, navigate to the "Datasets and Testing" section in the sidebar and select the dataset you want to configure the evaluator for. Click the Add Auto-Evaluator button at the top right of the dashboard to add an evaluator to the dataset. This will open a modal you can use to configure the evaluator.

Select the LLM-as-a-judge option and give your evaluator a name. You will now have the option to set an inline prompt or load a prompt from the prompt hub that will be used to evaluate the results of the runs in the experiment. For the sake of this example, choose the Create Few-Shot Evaluator option, as shown in Figure 10-11.

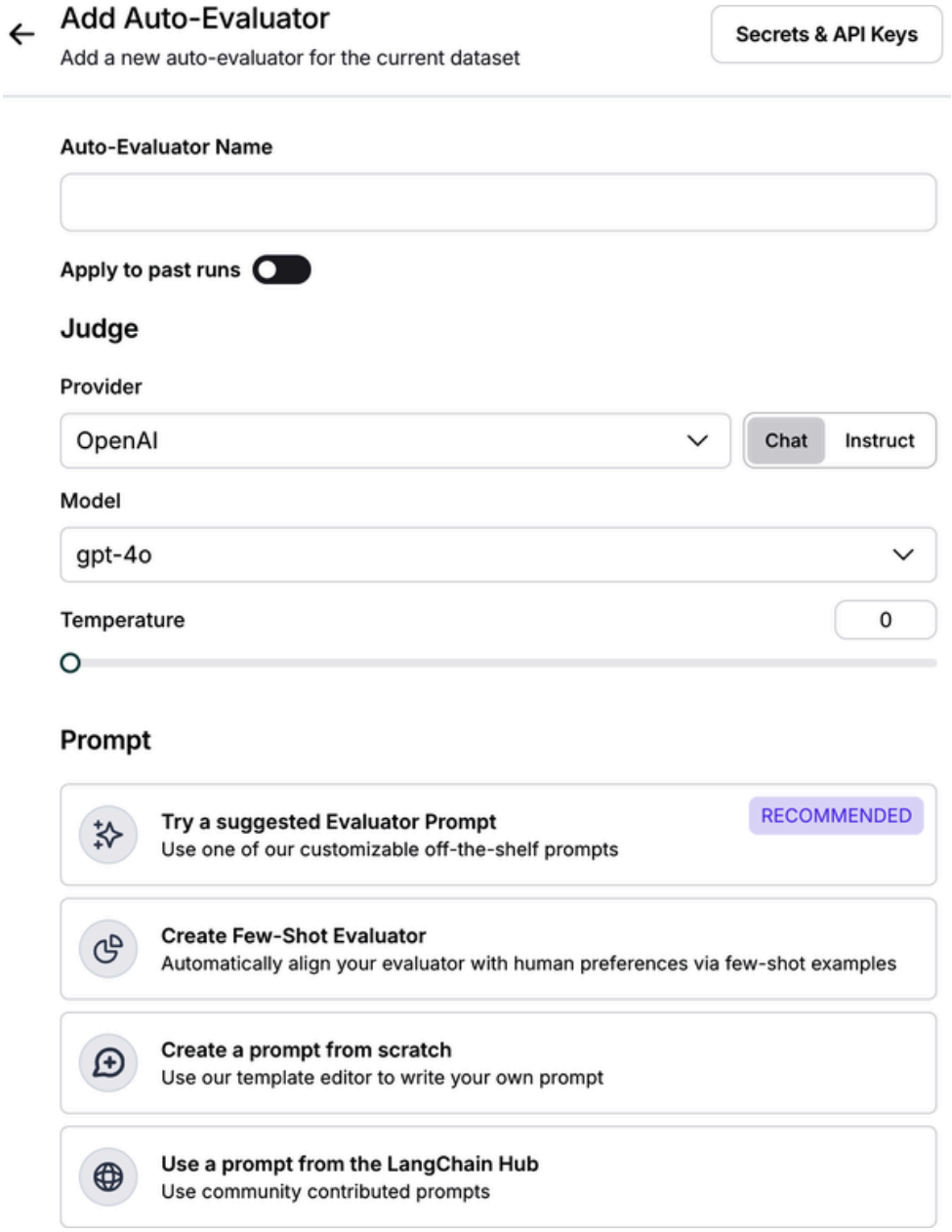

Figure 10-11. LangSmith UI options for the LLM-as-a-judge evaluator

This option will create a dataset that holds few-shot examples that will autopopulate when you make corrections on the evaluator feedback. The examples in this dataset will be inserted in the system prompt message.

You can also specify the scoring criteria in the Schema field and toggle between primitive types—for example, integer and Boolean (see Figure 10-12).

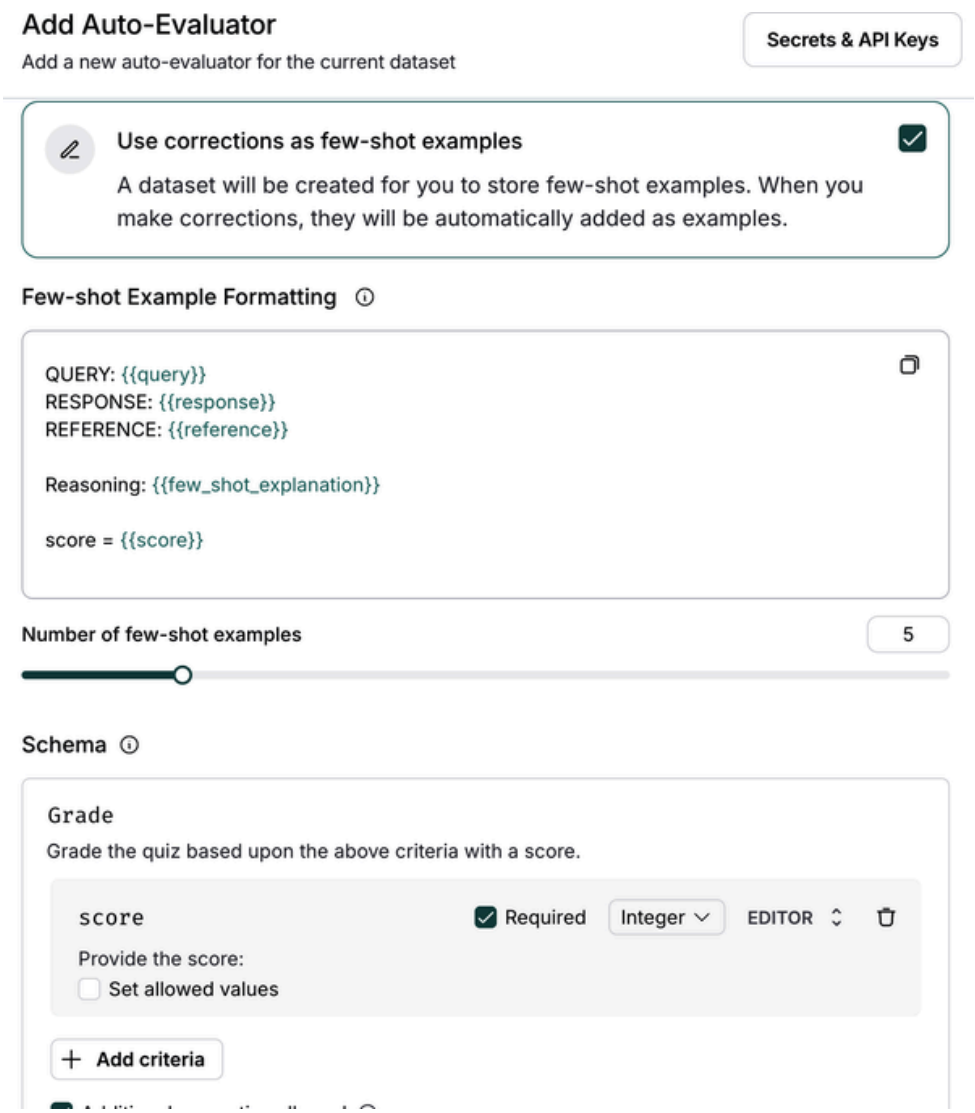

Figure 10-12. LLM-as-a-judge evaluator scoring criteria

Save the evaluator and navigate back to the dataset details page. Moving forward, each subsequent experiment run from the dataset will be evaluated by the evaluator you configured.

Pairwise evaluation

Ranking LLM outputs by preference can be less cognitively demanding for human or LLM-as-a-judge evaluators. For example, assessing which output is more informative, specific, or safe. Pairwise evaluation compares two outputs simultaneously from different versions of an application to determine which version better meets evaluation criteria.

LangSmith natively supports running and visualizing pairwise LLM app generations, highlighting preference for one generation over another based on guidelines set by the pairwise evaluator. LangSmith's pairwise evaluation enables you to do the following:

- Define a custom pairwise LLM-as-a-judge evaluator using any desired criteria
- Compare two LLM generations using this evaluator

As per the LangSmith docs, you can use custom pairwise evaluators in the LangSmith SDK and visualize the results of pairwise evaluations in the LangSmith UI.

After creating an evaluation experiment, you can navigate to the Pairwise Experiments tab in the Datasets & Experiments section. The UI enables you to dive into each pairwise experiment, showing which LLM generation is preferred based upon our criteria. If you click the RANKED_PREFERENCE score under each answer, you can dive deeper into each evaluation trace (see Figure 10-13).

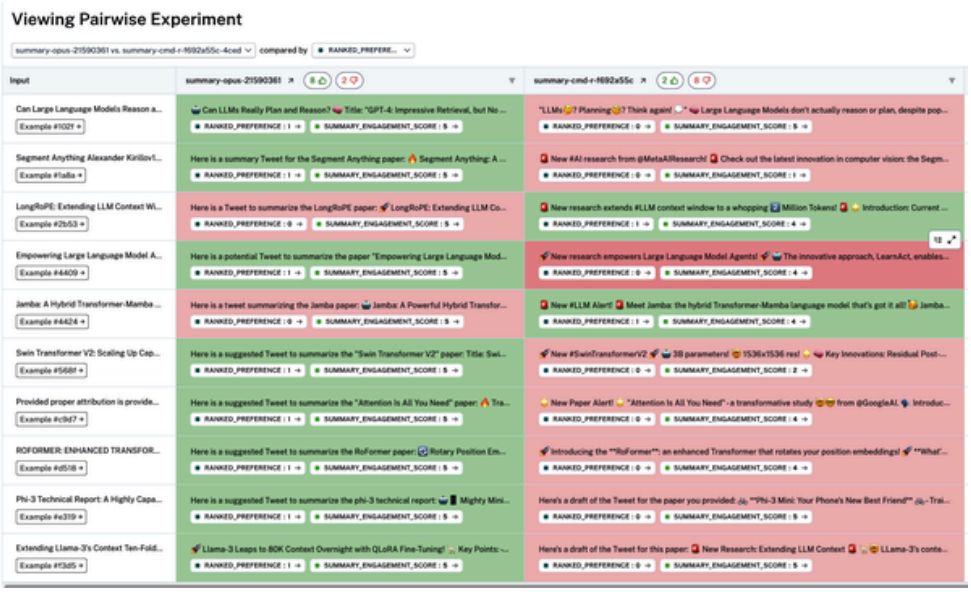

Figure 10-13. Pairwise experiment UI evaluation trace

Regression Testing

In traditional software development, tests are expected to pass 100% based on functional requirements. This ensures stable behavior once the test is validated. In contrast, however, AI models' output performances can vary significantly due to model *drift* (degradation due to changes in data distribution or updates to the model). As a result, testing AI applications may not always lead to a perfect score on the evaluation dataset.

This has several implications. First, it's important to track results and performance of your tests over time to prevent regression of your app's performance. *Regression* testing ensures that the latest updates or changes of the LLM model of your app do not *regress* (perform worse) relative to the baseline.

Second, it's crucial to compare the individual data points between two or more experimental runs to see where the model got it right or wrong.

LangSmith's comparison view has native support for regression testing, allowing you to quickly see examples that have changed relative to the baseline. Runs that regressed or improved are highlighted differently in the LangSmith dashboard (see Figure 10-14).

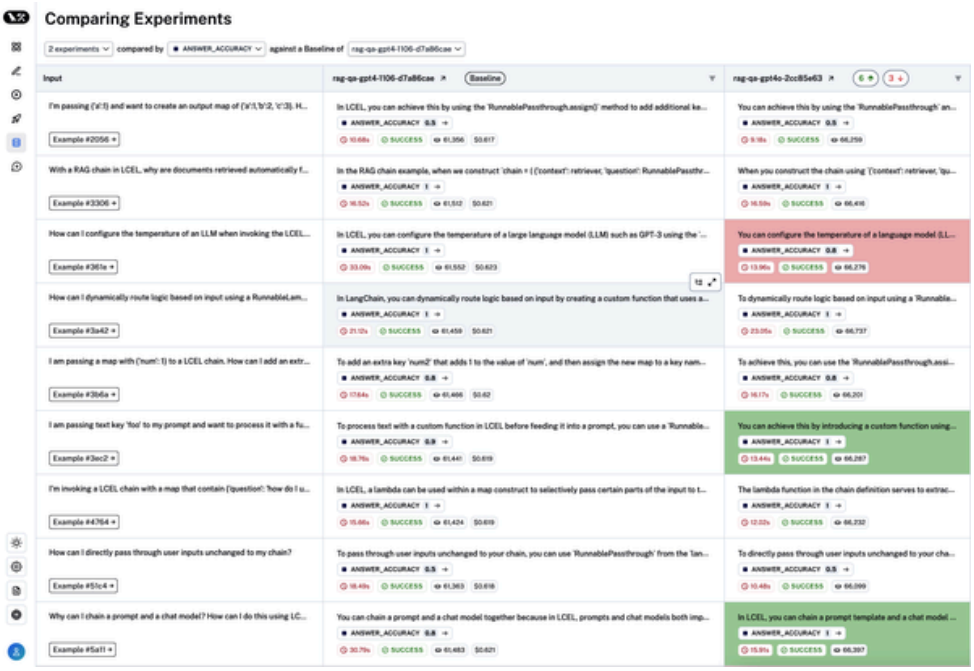

Figure 10-14. LangSmith's experiments comparison view

In LangSmith's Comparing Experiments dashboard, you can do the following:

- Compare multiple experiments and runs associated with a dataset. Aggregate stats of runs is useful for migrating models or prompts, which may result in performance improvements or regression on specific examples.
- Set a baseline run and compare it against prior app versions to detect unexpected regressions. If a regression occurs, you can isolate both the app version and the specific examples that contain performance changes.
- Drill into data points that behaved differently between compared experiments and runs.

This regression testing is crucial to ensure that your application maintains high performance over time regardless of updates and LLM changes.

Now that we've covered various preproduction testing strategies, let's explore a specific use case.

Evaluating an Agent's End-to-End Performance

Although agents show a lot of promise in executing autonomous tasks and workflows, testing an agent's performance can be challenging. In previous chapters, you learned how agents use tool calling with planning and memory to generate responses. In particular, tool calling enables the model to respond to a given prompt by generating a tool to invoke and the input arguments required to execute the tool.

Since agents use an LLM to decide the control flow of the application, each agent run can have significantly different outcomes. For example, different tools might be called, agents might get stuck in a loop, or the number of steps from start to finish can vary significantly.

Ideally, agents should be tested at three different levels of granularity:

Response

The agent's final response to focus on the end-to-end performance. The inputs are a prompt and an optional list of tools, whereas the output is the final agent response.

Single step

Any single, important step of the agent to drill into specific tool calls or decisions. In this case, the output is a tool call.

Trajectory

The full trajectory of the agent. In this case, the output is the list of tool calls.

Figure 10-15 illustrates these levels:

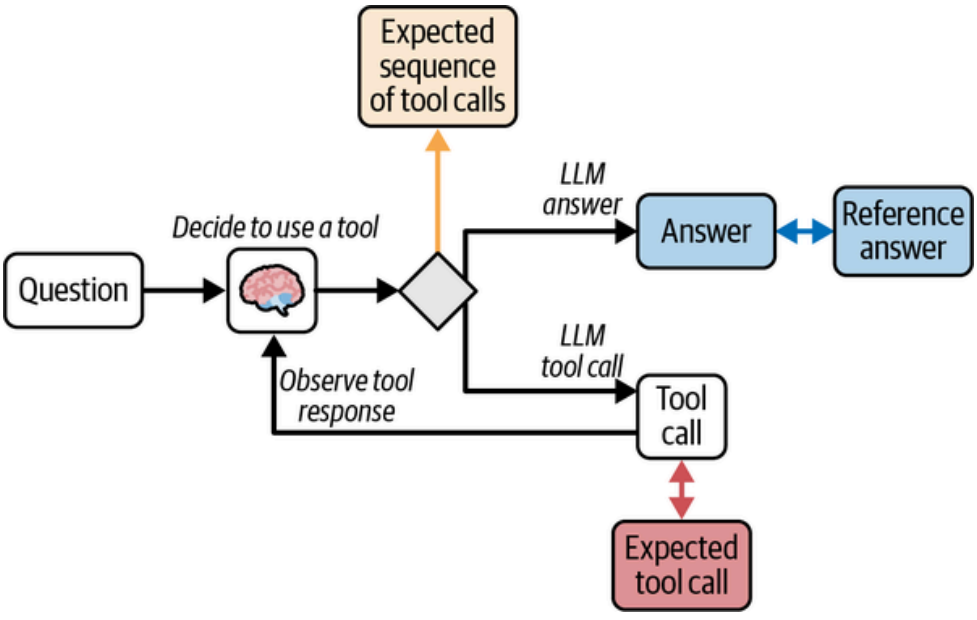

Figure 10-15. An example of an agentic app's flow

Let's dive deeper into each of these three agent-testing granularities.

Testing an agent's final response

In order to assess the overall performance of an agent on a task, you can treat the agent as a black box and define success based on whether or not it completes the task.

Testing for the agent's final response typically involves the following:

Inputs

User input and (optionally) predefined tools

Output

Agent's final response

Evaluator

LLM-as-a-judge

To implement this in a programmatic manner, first create a dataset that includes questions and expected answers from the agent:

Python

```
from langsmith import Client

client = Client()

# Create a dataset
examples = [
    ("Which country's customers spent the most? And how much did they spend
        """The country whose customers spent the most is the USA, with a to
        expenditure of $523.06"""),
    ("What was the most purchased track of 2013?",
        "The most purchased track of 2013 was Hot Girl."),
    ("How many albums does the artist Led Zeppelin have?",
        "Led Zeppelin has 14 albums"),
    ("What is the total price for the album “Big Ones”?",
        "The total price for the album 'Big Ones' is 14.85"),
    ("Which sales agent made the most in sales in 2009?",
        "Steve Johnson made the most sales in 2009"),
]

dataset_name = "SQL Agent Response"
if not client.has_dataset(dataset_name=dataset_name):
    dataset = client.create_dataset(dataset_name=dataset_name)
    inputs, outputs = zip(
        *[({"input": text}, {"output": label}) for text, label in examples
    )
```

```
    client.create_examples(inputs=inputs, outputs=outputs, dataset_id=datas

## chain
def predict_sql_agent_answer(example: dict):
    """Use this for answer evaluation"""
    msg = {"messages": ("user", example["input"])}
    messages = graph.invoke(msg, config)
    return {"response": messages['messages'][-1].content}
```

JavaScript

```
import { Client } from 'langsmith';

const client = new Client();

// Create a dataset
const examples = [
  ["Which country's customers spent the most? And how much did they spend?
    `The country whose customers spent the most is the USA, with a total
    expenditure of $523.06`],
  ["What was the most purchased track of 2013?",
    "The most purchased track of 2013 was Hot Girl."],
  ["How many albums does the artist Led Zeppelin have?",
    "Led Zeppelin has 14 albums"],
  ["What is the total price for the album 'Big Ones'?",
    "The total price for the album 'Big Ones' is 14.85"],
  ["Which sales agent made the most in sales in 2009?",
    "Steve Johnson made the most sales in 2009"],
];

const datasetName = "SQL Agent Response";

async function createDataset() {
  const hasDataset = await client.hasDataset({ datasetName });

  if (!hasDataset) {
    const dataset = await client.createDataset(datasetName);
    const inputs = examples.map(([text]) => ({ input: text }));
    const outputs = examples.map(([, label]) => ({ output: label }));

    await client.createExamples({ inputs, outputs, datasetId: dataset.id }
  }
}

createDataset();

// Chain function
async function predictSqlAgentAnswer(example) {
  // Use this for answer evaluation
  const msg = { messages: [{ role: "user", content: example.input }] };
  const output = await graph.invoke(msg, config);
```

```
  return { response: output.messages[output.messages.length - 1].content }
}
```

Next, as discussed earlier, we can utilize the LLM to compare the generated answer with the reference answer:

Python

```python
from langchain import hub
from langchain_openai import ChatOpenAI
from langsmith.evaluation import evaluate

# Grade prompt
grade_prompt_answer_accuracy = hub.pull("langchain-ai/rag-answer-vs-refere

def answer_evaluator(run, example) -> dict:
    """
    A simple evaluator for RAG answer accuracy
    """

    # Get question, ground truth answer, RAG chain answer
    input_question = example.inputs["input"]
    reference = example.outputs["output"]
    prediction = run.outputs["response"]

    # LLM grader
    llm = ChatOpenAI(model="gpt-4o", temperature=0)

    # Structured prompt
    answer_grader = grade_prompt_answer_accuracy | llm

    # Run evaluator
    score = answer_grader.invoke({"question": input_question,
                                  "correct_answer": reference,
                                  "student_answer": prediction})
    score = score["Score"]

    return {"key": "answer_v_reference_score", "score": score}


## Run evaluation
experiment_results = evaluate(
    predict_sql_agent_answer,
    data=dataset_name,
    evaluators=[answer_evaluator],
    num_repetitions=3,
)
```

JavaScript

```
import { pull } from "langchain/hub";
import { ChatOpenAI } from "langchain_openai";
import { evaluate } from "langsmith/evaluation";

async function answerEvaluator(run, example) {
  /**
   * A simple evaluator for RAG answer accuracy
   */

  // Get question, ground truth answer, RAG chain answer
  const inputQuestion = example.inputs["input"];
  const reference = example.outputs["output"];
  const prediction = run.outputs["response"];

  // LLM grader
  const llm = new ChatOpenAI({ model: "gpt-4o", temperature: 0 });

  // Grade prompt
  const gradePromptAnswerAccuracy = pull(
    "langchain-ai/rag-answer-vs-reference"
  );

  // Structured prompt
  const answerGrader = gradePromptAnswerAccuracy.pipe(llm);

  // Run evaluator
  const scoreResult = await answerGrader.invoke({
    question: inputQuestion,
    correct_answer: reference,
    student_answer: prediction
  });

  const score = scoreResult["Score"];

  return { key: "answer_v_reference_score", score: score };
}

// Run evaluation
const experimentResults = evaluate(predictSqlAgentAnswer, {
  data: datasetName,
  evaluators: [answerEvaluator],
  numRepetitions: 3,
});
```

Testing a single step of an agent

Testing an agent's individual action or decision enables you to identify and analyze specifically where your application is underperforming. Testing for a single step of an agent involves the following:

Inputs

User input to a single step (for example, user prompt, set of tools). This can also include previously completed steps.

Output

LLM response from the inputs step, which often contains tool calls indicating what action the agent should take next.

Evaluator

Binary score for correct tool selection and heuristic assessment of the tool input's accuracy.

The following example checks a specific tool call using a custom evaluator:

Python

```python
from langsmith.schemas import Example, Run

def predict_assistant(example: dict):
    """Invoke assistant for single tool call evaluation"""
    msg = [ ("user", example["input"]) ]
    result = assistant_runnable.invoke({"messages":msg})
    return {"response": result}


def check_specific_tool_call(root_run: Run, example: Example) -> dict:
    """
    Check if the first tool call in the response matches the expected tool
    """
    # Expected tool call
    expected_tool_call = 'sql_db_list_tables'

    # Run
    response = root_run.outputs["response"]

    # Get tool call
    try:
        tool_call = getattr(response, 'tool_calls', [])[0]['name']
    except (IndexError, KeyError):
        tool_call = None

    score = 1 if tool_call == expected_tool_call else 0
    return {"score": score, "key": "single_tool_call"}

experiment_results = evaluate(
    predict_assistant,
    data=dataset_name,
    evaluators=[check_specific_tool_call],
    num_repetitions=3,
```

```
    metadata={"version": metadata},
)
```

JavaScript

```
import {evaluate} from 'langsmith/evaluation';

// Predict Assistant
function predictAssistant(example) {
    /**
     * Invoke assistant for single tool call evaluation
     */
    const msg = [{ role: "user", content: example.input }];
    const result = assistantRunnable.invoke({ messages: msg });
    return { response: result };
}

// Check Specific Tool Call
function checkSpecificToolCall(rootRun, example) {
    /**
     * Check if the first tool call in the response matches the expected
     * tool call.
     */

    // Expected tool call
    const expectedToolCall = "sql_db_list_tables";

    // Run
    const response = rootRun.outputs.response;

    // Get tool call
    let toolCall;
    try {
        toolCall = response.tool_calls?.[0]?.name;
    } catch (error) {
        toolCall = null;
    }

    const score = toolCall === expectedToolCall ? 1 : 0;
    return { score, key: "single_tool_call" };
}

// Experiment Results
const experimentResults = evaluate(predictAssistant, {
    data: datasetName,
    evaluators: [checkSpecificToolCall],
    numRepetitions: 3,
});
```

The preceding code block implements these distinct components:

- Invoke the assistant, `assistant_runnable`, with a prompt and check if the resulting tool call is as expected.
- Utilize a specialized agent where the tools are hardcoded rather than passed with the dataset input.
- Specify the reference tool call for the step that we are evaluating for `expected_tool_call`.

Testing an agent's trajectory

It's important to look back on the steps an agent took in order to assess whether or not the trajectory lined up with expectations of the agent—that is, the number of steps or sequence of steps taken.

Testing an agent's trajectory involves the following:

Inputs

User input and (optionally) predefined tools.

Output

Expected sequence of tool calls or a list of tool calls in any order.

Evaluator

Function over the steps taken. To test the outputs, you can look at an exact match binary score or metrics that focus on the number of incorrect steps. You'd need to evaluate the full agent's trajectory against a reference trajectory and then compile as a set of messages to pass into the LLM-as-a-judge.

The following example assesses the trajectory of tool calls using custom evaluators:

Python

```python
def predict_sql_agent_messages(example: dict):
    """Use this for answer evaluation"""
    msg = {"messages": ("user", example["input"])}
    messages = graph.invoke(msg, config)
    return {"response": messages}

def find_tool_calls(messages):
    """
    Find all tool calls in the messages returned
    """
    tool_calls = [
        tc['name']
        for m in messages['messages'] for tc in getattr(m, 'tool_calls', [
    ]
```

```
    return tool_calls

def contains_all_tool_calls_any_order(
    root_run: Run, example: Example
) -> dict:
    """
    Check if all expected tools are called in any order.
    """
    expected = [
        'sql_db_list_tables',
        'sql_db_schema',
        'sql_db_query_checker',
        'sql_db_query',
        'check_result'
    ]
    messages = root_run.outputs["response"]
    tool_calls = find_tool_calls(messages)
    # Optionally, log the tool calls -
    #print("Here are my tool calls:")
    #print(tool_calls)
    if set(expected) <= set(tool_calls):
        score = 1
    else:
        score = 0
    return {"score": int(score), "key": "multi_tool_call_any_order"}

def contains_all_tool_calls_in_order(root_run: Run, example: Example) -> di
    """
    Check if all expected tools are called in exact order.
    """
    messages = root_run.outputs["response"]
    tool_calls = find_tool_calls(messages)
    # Optionally, log the tool calls -
    #print("Here are my tool calls:")
    #print(tool_calls)
    it = iter(tool_calls)
    expected = [
        'sql_db_list_tables',
        'sql_db_schema',
        'sql_db_query_checker',
        'sql_db_query',
        'check_result'
    ]
    if all(elem in it for elem in expected):
        score = 1
    else:
        score = 0
    return {"score": int(score), "key": "multi_tool_call_in_order"}

def contains_all_tool_calls_in_order_exact_match(
    root_run: Run, example: Example
) -> dict:
    """
```

```
    Check if all expected tools are called in exact order and without any
        additional tool calls.
    """
    expected = [
        'sql_db_list_tables',
        'sql_db_schema',
        'sql_db_query_checker',
        'sql_db_query',
        'check_result'
    ]
    messages = root_run.outputs["response"]
    tool_calls = find_tool_calls(messages)
    # Optionally, log the tool calls -
    #print("Here are my tool calls:")
    #print(tool_calls)
    if tool_calls == expected:
        score = 1
    else:
        score = 0

    return {"score": int(score), "key": "multi_tool_call_in_exact_order"}

experiment_results = evaluate(
    predict_sql_agent_messages,
    data=dataset_name,
    evaluators=[
        contains_all_tool_calls_any_order,
        contains_all_tool_calls_in_order,
        contains_all_tool_calls_in_order_exact_match
    ],
    num_repetitions=3,
)
```

JavaScript

```
import {evaluate} from 'langsmith/evaluation';

// Predict SQL Agent Messages
function predictSqlAgentMessages(example) {
  /**
   * Use this for answer evaluation
   */
  const msg = { messages: [{ role: "user", content: example.input }] };
  // Replace with your graph and config
  const messages = graph.invoke(msg, config);
  return { response: messages };
}

// Find Tool Calls
function findToolCalls({messages}) {
  /**
   * Find all tool calls in the messages returned
```

```
   */
  return messages.flatMap(m => m.tool_calls?.map(tc => tc.name) || []);
}

// Contains All Tool Calls (Any Order)
function containsAllToolCallsAnyOrder(rootRun, example) {
  /**
   * Check if all expected tools are called in any order.
   */
  const expected = [
    "sql_db_list_tables",
    "sql_db_schema",
    "sql_db_query_checker",
    "sql_db_query",
    "check_result"
  ];
  const messages = rootRun.outputs.response;
  const toolCalls = findToolCalls(messages);

  const score = expected.every(tool => toolCalls.includes(tool)) ? 1 : 0;
  return { score, key: "multi_tool_call_any_order" };
}

// Contains All Tool Calls (In Order)
function containsAllToolCallsInOrder(rootRun, example) {
  /**
   * Check if all expected tools are called in exact order.
   */
  const messages = rootRun.outputs.response;
  const toolCalls = findToolCalls(messages);

  const expected = [
    "sql_db_list_tables",
    "sql_db_schema",
    "sql_db_query_checker",
    "sql_db_query",
    "check_result"
  ];

  const score = expected.every(tool => {
    let found = false;
    for (let call of toolCalls) {
      if (call === tool) {
          found = true;
          break;
      }
    }
    return found;
  }) ? 1 : 0;

  return { score, key: "multi_tool_call_in_order" };
}
```

```
// Contains All Tool Calls (Exact Order, Exact Match)
function containsAllToolCallsInOrderExactMatch(rootRun, example) {
  /**
   * Check if all expected tools are called in exact order and without any
   * additional tool calls.
   */
  const expected = [
    "sql_db_list_tables",
    "sql_db_schema",
    "sql_db_query_checker",
    "sql_db_query",
    "check_result"
  ];
  const messages = rootRun.outputs.response;
  const toolCalls = findToolCalls(messages);

  const score = JSON.stringify(toolCalls) === JSON.stringify(expected)
    ? 1
    : 0;
  return { score, key: "multi_tool_call_in_exact_order" };
}

// Experiment Results
const experimentResults = evaluate(predictSqlAgentMessages, {
  data: datasetName,
  evaluators: [
    containsAllToolCallsAnyOrder,
    containsAllToolCallsInOrder,
    containsAllToolCallsInOrderExactMatch
  ],
  numRepetitions: 3,
});
```

This implementation example includes the following:

- Invoking a precompiled LangGraph agent `graph.invoke` with a prompt
- Utilizing a specialized agent where the tools are hardcoded rather than passed with the dataset input
- Extracting of the list of tools called using the function `find_tool_calls`
- Checking if all expected tools are called in any order using the function `contains_all_tool_calls_any_order` or called in order using `contains_all_tool_calls_in_order`
- Checking whether all expected tools are called in the exact order using `contains_all_tool_calls_in_order_exact_match`

All three of these agent evaluation methods can be observed and debugged in LangSmith's experimentation UI (see Figure 10-16).

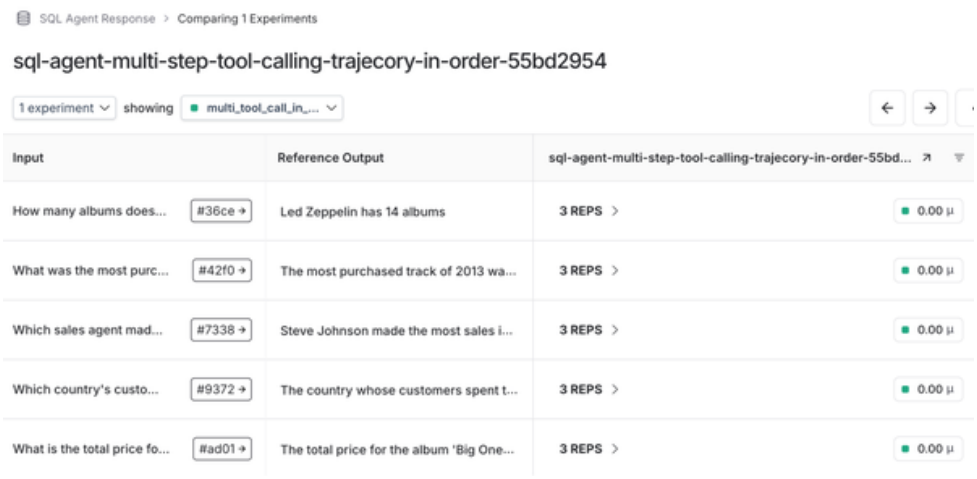

Figure 10-16. Example of an agent evaluation test in the LangSmith UI

In general, these tests are a solid starting point to help mitigate an agent's cost and unreliability due to LLM invocations and variability in tool calling.

Production

Although testing in the preproduction phase is useful, certain bugs and edge cases may not emerge until your LLM application interacts with live users. These issues can affect latency, as well as the relevancy and accuracy of outputs. In addition, observability and the process of *online evaluation* can help ensure that there are guardrails for LLM inputs or outputs. These guardrails can provide much-needed protection from prompt injection and toxicity.

The first step in this process is to set up LangSmith's tracing feature.

Tracing

A *trace* is a series of steps that your application takes to go from input to output. LangSmith makes it easy to visualize, debug, and test each trace generated from your app.

Once you've installed the relevant LangChain and LLM dependencies, all you need to do is configure the tracing environment variables based on your LangSmith account credentials:

```
export LANGCHAIN_TRACING_V2=true
export LANGCHAIN_API_KEY=<your-api-key>

# The below examples use the OpenAI API, though you can use other LLM provi

export OPENAI_API_KEY=<your-openai-api-key>
```

After the environment variables are set, no other code is required to enable tracing. Traces will be automatically logged to their specific project in the "Tracing projects" section of the LangSmith dashboard. The metrics provided include trace volume, success and failure rates, latency, token count and cost, and more—as shown in Figure 10-17.

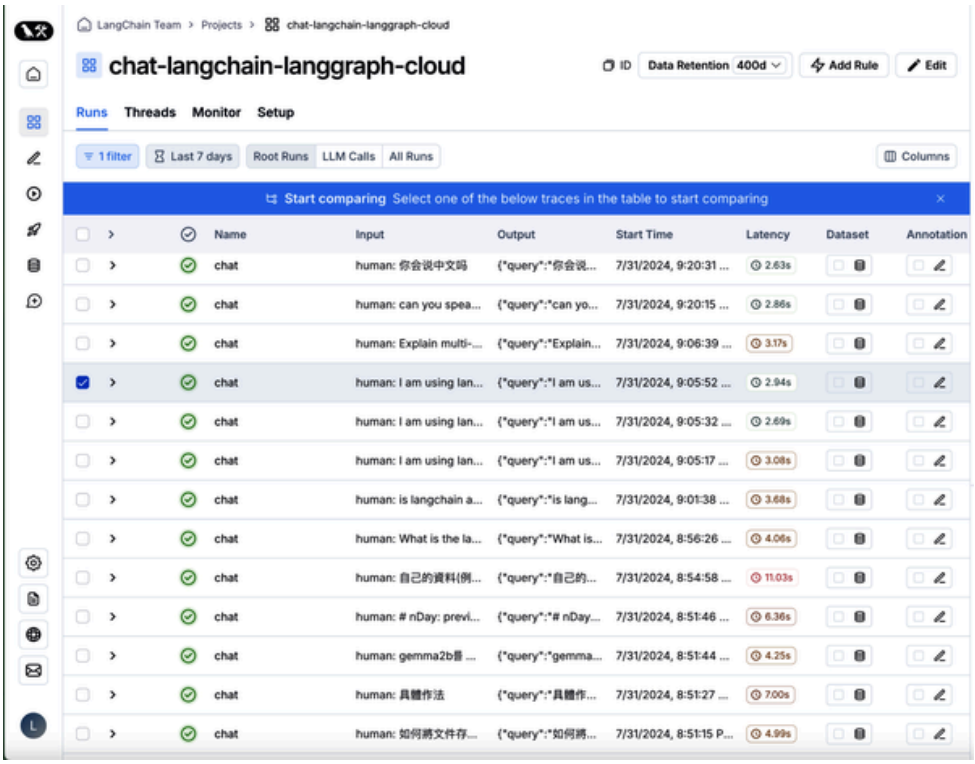

Figure 10-17. An example of LangSmith's trace performance metrics

You can review a variety of strategies to implement tracing based on your needs.

Collect Feedback in Production

Unlike the preproduction phase, evaluators for production testing don't have grounded reference responses for the LLM to compare against. Instead, evaluators need to score performance in real time as your application processes user inputs. This reference-free, real-time evaluation is often referred to as *online evaluation*.

There are at least two types of feedback you can collect in production to improve app performance:

Feedback from users

You can directly collect user feedback explicitly or implicitly. For example, giving users the ability to click a like and dislike button or provide detailed feedback based on the application's output is an effective way to track user satisfaction. In LangSmith, you can attach user feedback to any trace or intermediate run (that is, span) of a trace, including annotating traces inline or reviewing runs together in an annotation queue.

Feedback from LLM-as-a judge evaluators

As discussed previously, these evaluators can be implemented directly on traces to identify hallucination and toxic responses.

The earlier preproduction section already discussed how to set up LangSmith's auto evaluation in the Datasets & Experiments section of the dashboard.

Classification and Tagging

In order to implement effective guardrails against toxicity or gather insights on user sentiment analysis, we need to build an effective system for labeling user inputs and generated outputs.

This system is largely dependent on whether or not you have a dataset that contains reference labels. If you don't have preset labels, you can use the LLM-as-a-judge evaluator to assist in performing classification and tagging based upon specified criteria.

If, however, ground truth classification labels are provided, then a custom heuristic evaluator can be used to score the chain's output relative to the ground truth class labels.

Monitoring and Fixing Errors

Once your application is in production, LangSmith's tracing will catch errors and edge cases. You can add these errors into your test dataset for offline evaluation in order to prevent recurrences of the same issues.

Another useful strategy is to release your app in phases to a small group of beta users before a larger audience can access its features. This will enable you to uncover crucial bugs, develop a solid evaluation dataset with ground truth references, and assess the general performance of the app including cost, latency, and quality of outputs.

Summary

As discussed in this chapter, robust testing is crucial to ensure that your LLM application is accurate, reliable, fast, toxic-free, and cost-efficient. The three key stages of LLM app development create a data cycle that helps to ensure high performance throughout the lifetime of the application.

During the design phase, in-app error handling enables self-correction before the error reaches the user. Preproduction testing ensures each of

your app's updates avoids regression in performance metrics. Finally, production monitoring gathers real-time insights and application errors that inform the subsequent design process and the cycle repeats.

Ultimately, this process of testing, evaluation, monitoring, and continuous improvement, will help you fix issues and iterate faster, and most importantly, deliver a product that users can trust to consistently deliver their desired results.

Table of contents collapsed